Contemporary Mathematics
Volume **588**, 2013
http://dx.doi.org/10.1090/conm/588/11705

Modularity maximization in networks by variable neighborhood search

Daniel Aloise, Gilles Caporossi, Pierre Hansen, Leo Liberti, Sylvain Perron, and Manuel Ruiz

Abstract. Finding communities, or clusters, in networks, or graphs, has been the subject of intense studies in the last ten years. The most used criterion for that purpose, despite some recent criticism, is modularity maximization, proposed by Newman and Girvan. It consists in maximizing the sum for all clusters of the number of inner edges minus the expected number of inner edges assuming the same distribution of degrees. Numerous heuristics, as well as a few exact algorithms have been proposed to maximize modularity. We apply the Variable Neighborhood Search metaheuristic to that problem. Computational results are reported for the instances of the 10^{th} DIMACS Implementation Challenge. The algorithm presented in this paper obtained the second prize in the quality modularity (sub)challenge of the referred competition, finding the best known solutions for 11 out of 30 instances.

1. Introduction

Clustering is an important chapter of data analysis and data mining with numerous applications in natural and social sciences as well as in engineering and medicine. It aims at solving the following general problem: given a set of entities, find subsets, or clusters, which are homogeneous and/or well-separated. As the concepts of homogeneity and of separation can be made precise in many ways, there are a large variety of clustering problems [**HJ**, **JMF**, **KR**, **M**]. These problems in turn are solved by exact algorithms or, more often and particularly for large data sets, by heuristics, of which there are frequently a large variety. An exact algorithm provides, hopefully in reasonable computing time, an optimal solution together with a

1991 *Mathematics Subject Classification.* Primary 90C27, 90C59, 91C20.

Key words and phrases. Modularity, community, clustering.

The first author was partially supported by the National Council for Scientific and Technological Development - CNPq/Brazil grant number 305070/2011-8.

The second author was partially supported by FQRNT (Fonds de recherche du Québec – Nature et technologies) team grant PR-131365 and NSERC(Natural Sciences and Engineering Research Council of Canada) grant 298138-2009.

The third author was partially supported by FQRNT team grant PR-131365, NSERC grant 105574-07 and Digiteo grant *Senior Chair* 2009-14D "RMNCCO".

The fourth author was partially supported by Digiteo grant *Senior Chair* 2009-14D "RMNCCO".

The fifth author was partially supported by FQRNT team grant PR-131365 and NSERC grant 327435-06.

proof of its optimality. A heuristic provides, usually in moderate computing time, a near optimal solution or sometimes an optimal solution but without proof of its optimality.

In the last decade, clustering on networks, or graphs, has been extensively studied, mostly in the physics and computer science communities, with recently a few forays from operations research. Rather than using the term cluster, the words *module* or *community* are often adopted in the physics literature. We use below the standard notation and terminology for graphs, i.e, a graph $G = (V, E, \omega)$ is composed of a set V of n vertices v_j and a set E of m edges $e_{ij} = \{v_i, v_j\}$. These edges may be weighted by the function $\omega(\{u, v\})$. If they are unweighted $\omega(\{u, v\}) = 1$. A subgraph $G_C = (C, E_C, \omega)$ of a graph $G = (V, E, \omega)$ induced by a set of vertices $C \subseteq V$ is a graph with vertex set C and edge set E_C equal to all edges with both vertices in C. Such a subgraph corresponds to a cluster (or module, or community) and many heuristics aim at finding a partition $\mathcal{C}$ of V into pairwise disjoint nonempty subsets $V_1, V_2, \ldots, V_N$ inducing subgraphs of G and covering V. Various objective functions have been proposed for evaluating such a partition. Among the best known are *multiway cut* [**GH**], *normalized cut* [**SM**], *ratio cut* [**AY**] and *modularity* [**NG**]. Initially proposed by Girvan and Newman in 2002 [**GN**] as a stopping rule for a hierarchical divisive heuristic, modularity was considered later as an independent criterion allowing determination of optimal partitions as well as comparison between partitions obtained by various methods.

Modularity aims at finding a partition of V which maximizes the sum, over all modules, of the number of inner edges minus the expected number of such edges assuming that they are drawn at random with the same distribution of degrees as in G. The following precise definition of *modularity* is given in [**NG**]:

$$Q = \sum_{C \in \mathcal{C}} [a_C - e_C],$$

where a_C is the fraction of all edges that lie within module C and e_C is the expected value of the same quantity in a graph in which the vertices have the same expected degrees but edges are placed at random. A maximum value of Q near to 0 indicates that the network considered is close to a random one (barring fluctuations), while a maximum value of Q near to 1 indicates strong community structure. Observe that maximizing modularity gives an optimal partition together with the optimal number of modules.

Let the weight vertex function be defined as:

$$\omega(v) = \begin{cases} \displaystyle\sum_{\{u,v\} \in E} \omega(\{u, v\}) & \text{if } \{v, v\} \notin E \\ \displaystyle\sum_{\{u,v\} \in E, u \neq v} \omega(\{u, v\}) + 2\omega(\{v, v\}) & \text{if } \{v, v\} \in E. \end{cases}$$

The modularity for a module C may be written as

$$Q(C) = \frac{\displaystyle\sum_{\{u,v\} \in E_C} \omega(\{u, v\})}{\displaystyle\sum_{e \in E} \omega(e)} - \frac{\left(\displaystyle\sum_{v \in V_C} \omega(v)\right)^2}{4\left(\displaystyle\sum_{e \in E} \omega(e)\right)^2}. \tag{1}$$

Let $\mathcal{C}$ be a partition of V. The sum over modules of their modularities can be written as

$$Q = \frac{\sum_{C\in\mathcal{C}} \sum_{\{u,v\}\in E_C} \omega(\{u,v\})}{\sum_{e\in E}\omega(e)} - \frac{\sum_{C\in\mathcal{C}} \left(\sum_{v\in V_C}\omega(v)\right)^2}{4\left(\sum_{e\in E}\omega(e)\right)^2}. \tag{2}$$

Numerous heuristics have been proposed to maximize modularity. They are based on divisive hierarchical clustering, agglomerative hierarchical clustering, partitioning, and hybrids. They rely upon various criteria for agglomeration or division [**BGLL**, **CNM**, **DDA**, **N04**, **WT**], simulated annealing [**GA**, **MD**, **MAD**], mean field annealing [**LH**], genetic search [**THB**], extremal optimization [**DA**], label propagation [**BC**, **LM**], spectral clustering [**N06**, **RMP**, **SDJB**], linear programming followed by randomized rounding [**AK**], dynamical clustering [**BILPR**], multilevel partitioning [**D**], contraction-dilation [**MHSWL**], multistep greedy search [**SC**], quantum mechanics [**NHZW**] and other approaches [**BGLL**, **CFS**, **FLZWD**, **KSKK**, **RZ**, **SDJB**]. For a more detailed survey, see [**F**]. While other metaheuristics have been applied to modularity maximization, it is the first time, to the best of our knowledge, that the Variable Neighborhood Search (VNS) metaheuristic is used for that purpose. In particular, by using decomposition, we were able to tackle larger problems than the previous metaheuristic approaches, reducing the size of the problem in which VNS is applied.

The paper is organized as follows: in Section 2, after giving an outline of the VNS metaheuristic, we discuss its application to modularity maximization. In Section 3, we recall and extend to the weighted case an exact method for modularity maximization which is used to evaluate the quality of the solutions obtained by our variable neighborhood search metaheuristic. Experimental results are presented in Section 4 in two tables corresponding to the results for Pareto and Quality challenges respectively. Brief conclusions are drawn in the last section.

2. Description of the heuristic

2.1. Outline of the variable neighborhood search metaheuristic. Variable Neighborhood Search (VNS) is a metaheuristic, or framework for building heuristics, aimed at solving combinatorial and global optimization problems. Since its inception, VNS has undergone many developments and has been applied in numerous fields (see [**HMP**] for a recent survey).

Metaheuristics address the problem of escaping, as much as possible, from local optima. A local maximum x_L of an optimization problem is such that

$$f(x_L) \geq f(x), \forall x \in N(x_L) \tag{3}$$

where $N(x)$ denotes the feasible *neighborhood* of x, which can be defined in many different ways each one yielding a different neighborhood structure. In discrete optimization problems, a neighborhood structure consists of all vectors obtained from x by some simple modification. For instance, for x binary, one neighborhood structure can be defined by the set of all vectors obtained from x by complementing one of its components. Another possible neighborhood structure can be defined as the set of all vectors obtained from x by complementing two complementary

components of x (i.e., one component is set from 0 to 1 and the other goes from 1 to 0). A *local search* or *improving* heuristic consists of choosing an initial solution x, and then moving to the best neighbor $x' \in N(x)$ in the case $f(x') > f(x)$. If no such neighbor exists, the heuristic stops, otherwise it is iterated.

If many local maxima exist for a problem, the range of values they span may be large. Moreover, the globaly optimum value $f(x^*)$ may differ substantially from the average value of a local maximum, or even from the best such value among many, obtained by some simple randomized heuristic. In order to escape from local maxima and, more precisely, the mountains of which they are the top, VNS exploits the idea of *neighborhood* change. In fact, VNS relies upon the following observations:

Fact 1: *A local maximum with respect to one neighborhood structure is not necessarily so for another*;

Fact 2: *A global maximum is a local maximum with respect to all possible neighborhood structures*;

Fact 3: *For many problems local maxima with respect to one or several neighborhoods are relatively close to each other.*

Let us denote with $\mathcal{N}_t$, $(t = 1, \ldots, t_{max})$, a finite set of pre-selected neighborhood structures, and with $\mathcal{N}_t(x)$ the set of solutions in the t^{th} neighborhood of x. We call x a *local maximum* with respect to $\mathcal{N}_t$ if there is no solution $x' \in \mathcal{N}_t(x)$ such that $f(x') > f(x)$.

In the VNS framework, the neighborhoods used correspond to various types of moves, or perturbations, of the current solution, and are problem specific. The current best solution x found is the center of the search. When looking for a better one, a solution x' is drawn at random in an increasingly far neighborhood and a local ascent is performed from x', leading to another local maximum x''. If $f(x'') \leq f(x)$, x'' is ignored and one chooses a new neighbor solution x' in a further neighborhood of x. If, otherwise, $f(x'') > f(x)$, the search is re-centered around x'' restarting with the closest neighborhood. If all neighborhoods of x have been explored without success, one begins again with the closest one to x, until a stopping condition (e.g. maximum CPU time) is satisfied.

As the size of neighborhoods tends to increase with their distance from the current best solution x, close-by neighborhoods are explored more thoroughly than far away ones. This strategy takes advantage of the three Facts 1–3 mentioned above. Indeed it is often observed that most or all local maxima of combinatorial problems are concentrated in a small part of the solution space. Thus, finding a first local maximum x implies that some important information has been obtained: to get a better, near-optimal solution, one should first explore its vicinity.

The algorithm proposed in this work has two main components: (i) an improvement heuristic, and (ii) exploration of different types of neighborhoods for getting out of local maxima. They are used within a variable neighborhood decomposition search framework [**HMP**] which explores the structure of the problem concentrating on small parts of it. The basic components as well as the decomposition framework are described in the next sections.

2.2. Improvement heuristic. The improvement heuristic we used is the LPAm+ algorithm proposed by Liu and Murata in [**LM**]. LPAm+ is composed of a label propagation algorithm proposed by Barber and Clark [**BC**] and a community merging routine. A strong feature of this heuristic is that label propagation

executes in near linear time (in fact, each iteration of label propagation executes in time proportional to m), while one round of merging pairs of communities can execute in $O(m \log n)$ [**SC**].

Label propagation is a similarity-based technique in which the label of a vertex is propagated to adjacent vertices according to their proximity. Label propagation algorithms for clustering problems assume that the label of a node correspond to its incumbent cluster index. Then, at each label propagation step, each vertex is sequentially evaluated for label updating according to a propagation rule. In [**BC**], the Barber and Clark propose a label propagation algorithm, called LPA, for modularity maximization. Their label updating rule for vertex v is (see. [**BC**] for details):

$$\ell_v \leftarrow \underset{\ell}{\operatorname{argmax}} \left(\sum_{u=1}^{n} B_{uv} \delta(\ell_u, \ell) \right) \tag{4}$$

where ℓ_v is the label of vertex v, $B_{uv} = \omega(\{u, v\}) - (\omega(u)\omega(v))/2m$, and $\delta(i, j)$ is the Kronecker's delta.

Moreover, the authors prove that the candidate labels for ℓ_v in eq.(4) can be confined to the labels of the vertices adjacent to v and an unused label. We decided to use this fact in order to speedup LPA. Let us consider a vertex $v^* \in C$, where C is a module of the current partition, and let us suppose that the modules to which its adjacent vertices belong have not changed since the last evaluation of v^*. In this case, v^* can be discarded for evaluation since no value has changed from the last instantiation of eq.(4) since the last evaluation of v^*. With that in mind, we decided to iterate over "labels" instead of over the vertices of the graph.

We used LPAm+ modified as follows. A list L of all labels is initialized with the clusters indices of the current partition. Then, from L, we proceed by picking a label $\ell \in L$ until L is empty. Each time a label ℓ is removed from L, we evaluate by means of eq.(4) all its vertices for label updating. If the label of a vertex is updated, yielding an improvement in the modularity value of the current partition, the old and the new labels of that vertex, denoted ℓ^{old} and ℓ^{new}, are inserted in L. Moreover, the labels of vertices which are connected to a node with label equal to either ℓ^{old} or ℓ^{new} are also inserted in L. This modification induces a considerable algorithmic speedup since only a few labels need to be evaluated as the algorithm proceeds.

We then tested this modified LPAm+, and proceeded to improve it based on empirical observations. In the final version, whenever a vertex relabeling yields an improvement, the old and the new labels of that vertex are added to L but only together with the labels of vertices which are adjacent to the relabeled vertex. This version was selected to be used in our experiments due to its benefits in terms of computing times and modularity maximization.

2.3. Neighborhoods for perturbations. In order to escape from local maxima, our algorithm uses five distinct neighborhoods for perturbing a solution. They are:

(1) SINGLETON: all the vertices in a cluster are made singleton clusters.
(2) DIVISION: splits a community into two equal parts. Vertices are assigned to each part randomly.

(3) NEIGHBOR: relabels each vertex of a cluster to one of the labels of its neighbors or to an unused label.
(4) EDGE: puts two linked vertices assigned to different clusters into one neighboring cluster randomly chosen.
(5) FUSION: merges two or more clusters into a single one.
(6) REDISTRIBUTION: destroys a cluster and spreads each one of its vertices to a neighboring cluster randomly chosen.

2.4. Variable Neighborhood Decomposition Search. Given the size of the instances proposed in the 10^{th} DIMACS Implementation Challenge, a decomposition framework was used. It allows the algorithm to explore the search space more quickly since just a small part of the solution is searched for improvement at a time. This subproblem is isolated for improvement through selecting a subset of the clusters in the incumbent solution.

The decomposition proposed here is combined with the five neighborhoods presented in the previous section within a variable neighborhood schema. Thus, the decomposition executes over five distinct neighborhood topologies, with subproblems varying their size according to the VNS paradigm. The pseudo-code of the variable neighborhood decomposition search heuristic is given in Figure 1.

```
1  Algorithm VNDS(P)
2  Construct a random solution x ;
3  x ← LPAm+(x, P) ;
4  s ← 1;
5  while stopping condition not satisfied do
6  |  Construct a subproblem S from x with a randomly selected
   |  cluster and s − 1 neighboring clusters ;
7  |  Select randomly
   |  α ∈ {singleton, division, neighbor, fusion, redistribution} ;
8  |  x′ ← shaking(x, α, S);
9  |  x′ ← LPAm+(x′, S) ;
10 |  if cost(x′) > cost(x) then
11 |  |  x ← LPAm(x′, P) ;
12 |  |  s ← 1;
13 |  else
14 |  |  s ← s + 1;
15 |  |  if s > min{MAX_SIZE, #clusters(x)} then
16 |  |  |  s ← 1;
17 |  |  end
18 |  end
19 end
20 return x
```

Algorithm 1: Pseudo-code of the decomposition heuristic.

The algorithm VNDS starts with a random solution for an input problem P in line 2. Then, in line 3 this solution is improved by applying our implementation of LPAm+. Note that LPAm+ receives two input parameters, they are: (i) the solution to be improved, and (ii) the space on which an improvement will be searched. In line 3, the local search is applied in the whole problem space P, which means that

all vertices are tested for label updating, and all clusters are considered for merging. In line 4, the variable s which controls the current decomposition size is set to 1.

The central part of the algorithm VNDS consists of the loop executed in lines 5-19 until a stopping criterion is met (this can be the number of non improving iterations for the Pareto Challenge or maximum allowed CPU time for the Quality Challenge). This loop starts in line 6 by constructing a subproblem from a randomly selected cluster and $s-1$ neighboring clusters. Then, in line 7 a neighborhood α is randomly selected for perturbing the incumbent solution x. Our algorithm allows choosing α by specifying a probability distribution on the neighborhoods. Thus, the most successful neighborhoods are more often selected. The shaking routine is actually performed in line 8 in the chosen neigborhood α and in the search space defined by subproblem S. In the following, the improving heuristic LPAm+ is applied over x' in line 9 only in the current subproblem S. If the new solution x' is better than x in line 10, a faster version of the improving heuristic, denoted LPAm, is applied in line 11 over x' in the whole problem P. In this version, the improving heuristic does not evaluate merging clusters. The resulting solution of LPAm application is assigned to x in line 11 and s is reset to 1 in line 12. Otherwise, if x' is not better than x, the size of the decomposition is increased by one in line 14. This value is reset to 1 in line 16 if it exceeds the minimum between a given parameter MAX_SIZE and the number of clusters (i.e., #clusters(x)) in the current solution x (line 15). Finally, a solution x is returned by the algorithm in line 20.

3. Description of the exact method

Column generation together with branch-and-bound can be used to obtain an optimal partition. Column generation algorithms for clustering implicitly take into account all possible communities (or, in other words, all subsets of the set of entities under study). They replace the problem of finding simultaneously all communities in an optimal partition by a sequence of optimization problems for finding one community at a time, or more precisely and for the problem under study a community which improves the modularity of the current solution. In [**ACCHLP**], several stabilized column generation algorithms have been proposed for modularity maximization and compared on a series of well-known problems from the literature. The column generation algorithm based on extending the mixed integer formulation of Xu et al. [**XTP**] appears to be the most efficient. We summarize below an adaptation of this algorithm for the case of weighted networks.

Column generation is a powerful technique of linear programming which allows the exact solution of linear programs with a number of columns exponential in the size of the input. To this effect, it follows the usual steps of the simplex algorithm, apart from finding an entering column with a positive reduced cost in case of maximization which is done by solving an auxiliary problem. The precise form of this last problem depends on the type of problem considered. It is often a combinatorial optimization or a global optimization problem. It can be solved heuristically as long as a column with a reduced cost of the required sign can be found. When this is no longer the case, an exact algorithm for the auxiliary problem must be applied either to find a column with the adequate reduced cost sign, undetected by the heuristic, or to prove that there remains no such column and hence the linear programming relaxation is solved.

For modularity maximization clustering, as for other clustering problems with an objective function additive over the clusters, the columns correspond to the set T of all subsets of V, i.e., to all nonempty modules, or in practice to a subset T' of T. To express this problem, define $a_{it} = 1$ if vertex i belongs to module t and $a_{it} = 0$ otherwise. One can then write the model as

$$\max \sum_{t \in T} c_t z_t \tag{5}$$

$$\text{s.t.} \sum_{t \in T} a_{it} z_t = 1 \qquad \forall i = 1, \ldots, n \tag{6}$$

$$z_t \in \{0, 1\} \qquad \forall t \in T, \tag{7}$$

where c_t corresponds to the modularity value of the module indexed by t with $t = 1 \ldots 2^n - 1$. The problem (5)-(7) is too large to be written explicitly. A reduced problem with few columns, i.e., those with index $t \in T'$, is solved instead. One first relaxes the integrality constraints and uses column generation for solving the resulting linear relaxation.

The auxiliary problem, for the weighted case, can be written as follows:

$$\begin{aligned} \max_{x \in \mathbb{B}^m, D \in \mathbb{R}} \quad & \sum_e \in E \frac{x_e}{M} - \left(\frac{D}{2M}\right)^2 - \sum_{u \in V} \lambda_u y_u \\ \text{s.t.} \quad & D = \sum_{u \in V} \omega(u) y_u \\ & x_e \leq y_u \qquad \forall e = \{u, v\} \in E \\ & x_e \leq y_v \qquad \forall e = \{u, v\} \in E \end{aligned}$$

where $M = \sum_{e \in E} \omega(e)$. Variable x_e is equal to 1 if edge e belongs to the community which maximizes the objective function and to 0 otherwise. Similarly, y_u is equal to 1 if the vertex u belongs to the community and 0 otherwise. The objective function is equal to the modularity of the community to be determined minus the scalar product of the current value λ_u of the dual variables times the indicator variables y_u. As in [**ACCHLP**], the auxiliary problem is first solved with a VNS heuristic as long as a column with a positive reduced cost can be found. When this is no more the case, CPLEX is called to find such a column or prove that none remain. If the optimal solution of the linear relaxation is not integer, one proceeds to branching on the condition that two selected entities belong to the same community or to two different ones.

4. Experimental Results

The algorithms were implemented in C++ and compiled by gcc 4.5.2. Limited computational experiments allowed to set the parameters of the VNDS algorithm as follows:

- MAX_SIZE = 15
- Probability distribution for selecting α is drawn with:
  - 30% of chances of selecting SINGLETON
  - 30% of chances of selecting DIVISION
  - 28% of chances of selecting NEIGHBOR
  - 5% of chances of selecting EDGE
  - 4% of chances of selecting FUSION

– 3% of chances of selecting REDISTRIBUTION

The stopping condition in algorithm VNDS was defined depending on the challenge, Pareto or Quality, in which VNDS is used. Thus, the same algorithm is able to compete in both categories by just modifying how it is halted.

4.1. Results for exactly solved instances. Exact algorithms provide a benchmark of exactly solved instances which can be used to fine tune heuristics. More precisely, the comparison of the symmetric differences between the optimal solution and the heuristically obtained ones may suggest additional moves which improve the heuristic under study.

In general, a sophisticated heuristic should be able to find quickly an optimal solution for most or possibly all practical instances which can be solved exactly with a proof of optimality. Our first set of experiments aims to verify the effectiveness of the VNDS algorithm in the instances for which the optimal solution is proved by the exact method of Section 3. The instances tested here are taken from the *Clustering* chapter of the 10^{th} DIMACS Implementation Challenge (http://www.cc.gatech.edu/dimacs10/archive/clustering.shtml).

Table 1 presents average solution values and CPU times (in seconds) obtained from five independent runs of the VNDS algorithm in a Intel X3353 with a 2.66 Ghz clock and 24Gb of RAM memory. The first column refers to the instance. The second and third columns refer to the number of nodes (n) and edges (m) of each instance. Fourth and fifth column show the VNDS average results. Finally, the sixth and seventh column present the optimal solution values proved by the exact algorithm.

TABLE 1. VNDS average results and optimal modularity values obtained by the exact algorithm of Section 3.

instance	n	m	Q_{avg}	t_{avg}	Q_{opt}	$\|\mathcal{C}\|_{opt}$
`karate`	34	78	0.419790	0.00	0.419790	4
`chesapeake`	39	170	0.265796	0.00	0.265796	3
`dolphins`	62	159	0.528519	0.00	0.528519	5
`lesmis`	77	254	0.566688	0.00	0.566688	6
`polbooks`	105	441	0.527237	0.00	0.527237	5
`adjnoun`	112	425	0.312268	0.09	0.313367	7
`football`	115	613	0.604570	0.00	0.604570	10
`jazz`	198	2742	0.445144	0.01	0.445144	4

We note that VNDS finds the optimal solutions of instances `karate`, `chesapeake`, `dolphins`, `lesmis`, `polbooks`, `adjnoun`, `football`, and `jazz`. Except for instance `adjnoun`, where the optimal solution is found in 2 out of 5 runs, the optimal solutions of the aforementioned instances are obtained in all runs.

4.2. Results for Pareto Challenge. The results presented in this section and in the following one refers to the final modularity instances of the 10^{th} DIMACS Implementation Challenge. Particularly for this section, results are presented both in terms of modularity values and CPU times, which are the two performance dimensions evaluated in the Pareto challenge. Computational experiments were performed on a Intel X3353 with a 2.66 Ghz clock and 24Gb of RAM memory.

Instances `kron_g500-simple-logn20` ($n = 1048576, m = 44619402$), `cage15` ($n = 5154859, m = 47022346$), `uk-2002` ($n = 18520486, m = 261787258$), `uk-2007-05` ($n = 105896555, m = 3301876564$), and `Er-fact1.5-scale25` ($\log_2 n = 25$) were not executed due to memory limitations.

In this challenge, VNDS stops whenever it attains either N iterations without improving the incumbent solution or after 3 hours of CPU. For this challenge we divided the instances into two categories. For the instances in category $P1$, VNDS uses $N = 1000$, while for those in category $P2$, the algorithm uses $N = 100$.

Table 2 shows average computational results obtained in five independent runs of algorithm VNDS. The results for one of these runs was sent to the DIMACS Pareto challenge. The first column in the table refers to the category of the instance indicated in the second column. The third and fourth columns refer to the number of nodes (n) and edges (m) of each instance. The fifth and sixth columns refer to average modularity values and computing times (in seconds), respectively. VNDS is stopped due to CPU time limit in the instances for which the average computing time $t_{avg} = 10800.00$.

We remark from Table 2:

- Instances `coPapersDBLP`, `audikw1`, and `ldoor` are stopped due to our CPU time limit in each one of the five independent runs.
- Considering all the instances presented in the table, VNDS was Pareto dominated (see http://www.cc.gatech.edu/dimacs10/data/dimacs10-rules.pdf) in the DIMACS challenge by at most 2 other algorithms in each instance. It is worthy mentioning that the organizing committee left open to the Pareto challenge participants the task of defining their own stopping condition for their algorithms. Consequently, Pareto challenge scores were sensitive to the strategy used by each team. For instance, considering only the instances we categorized in $P1$, which uses $N = 1000$ as stopping condition, VNDS was dominated by at most 1 other algorithm.
- In 9 out of 25 instances in the table, VNDS was not Pareto dominated by any other algorithm in the Pareto challenge.

4.3. Results for Quality challenge. Since the amount of work to compute a solution is not taken into consideration for this challenge, the VNDS algorithm was allowed to run for a longer period of time than before, the CPU time limit being the unique stopping condition. In our set of experiments, the instances were split into two different categories. The algorithm was allowed to run for 1800 seconds (30 minutes) for instances in category Qu1, and 18000 seconds (5 hours) for instances in category Qu2. Furthermore, in order to overcome memory limitations, VNDS was executed in a Intel Westmere-EP X5650 with a 2.66 Ghz clock and 48Gb of RAM memory for the largest instances. This allowed the algorithm to obtain solutions for instances `kron_g500-simple-logn20`, `cage15` and `uk-2002`.

Table 3 presents the computational results obtained in 10 independent runs of algorithm VNDS. We chose to present here the same results submitted to the DIMACS Implementation Challenge. The first column refers to the category of the instance indicated in the second column. Third and fourth columns refer to the best obtained modularity value and its corresponding number of clusters. Finally, the last column shows the rank position of the referred solution among 15 participating teams.

TABLE 2. Average modularity results for the Pareto Challenge of the 10^{th} DIMACS Implementation Challenge.

category	instance	n	m	Q_{avg}	t_{avg}
P1	celegans_metabolic	453	2025	0.452897	1.97
	e-mail	1133	5451	0.427105	1.69
	polblogs	1490	16715	0.582510	4.96
	power	4941	6594	0.940460	11.87
	PGPgiantcompo	10680	24316	0.885451	28.03
P2	astro-ph	16726	47594	0.738435	10.53
	memplus	17758	54196	0.686220	78.44
	as-22july06	22963	48436	0.676091	31.01
	cond-mat-2005	40421	175691	0.739293	106.54
	kron_g500-simple-logn16	65536	2456071	0.061885	12.91
	preferentialAttachment	100000	499985	0.314690	1972.50
	smallworld	100000	499998	0.791905	33.82
	G_n_pin_pout	100000	501198	0.488209	2625.18
	luxembourg.osm	114599	119666	0.989307	90.30
	rgg_n_2_17_s0	131072	728753	0.977417	214.53
	caidaRouterLevel	192244	609066	0.867259	2037.24
	coAuthorsCiteseer	227320	814134	0.901432	1754.33
	citationCiteseer	268495	1156647	0.820164	9048.09
	coPapersDBLP	540486	15245729	0.862878	10800.00
	eu-2005	862664	16138468	0.941157	9914.73
	audikw1	943695	38354076	0.918034	10800.00
	ldoor	952203	22785136	0.968844	10800.00
	in-2004	1382908	13591473	0.980509	9583.98
	belgium.osm	1441295	1549970	0.994569	6549.13
	333SP	3712815	11108633	0.988247	10019.93

A few remarks are in order regarding the results shown in Table 3:

- The VNDS algorithm obtained the best solutions for 11 out of 30 instances of the modularity challenge (i.e., approx. 37% of the instances). Moreover, the algorithm figured in the first two positions in 25 out of 30 instances (i.e., approx. 83%).
- The algorithm does not appear to have its effectiveness influenced by the number of clusters. For example, the algorithm found the best solution for instance celegans_metabolic using 9 clusters as well as it obtained the best result for instance kron_g500-simple-logn16 using 10027 clusters.
- The algorithm was particularly bad for instance cage15. Actually, the result submitted to the challenge corresponded to a simple LPAm+ application. This was due to two combined reasons: (i) the instance was large with $n = 5154859$ nodes and $m = 47022346$, and (ii) LPAm+ did not get to decrease much the number of clusters (648819). This led to algorithm abortion before executing its main computing part.

5. Conclusion

Several integer programming approaches and numerous heuristics have been applied to modularity maximization. They are due mostly to the physics and computer sciences research communities. We have applied the variable neighborhood

TABLE 3. Modularity results for the Quality Challenge of the 10^{th} DIMACS Implementation Challenge.

category	instance	Q_{best}	$\lvert\mathcal{C}\rvert_{best}$	rank
Qu1	celegans_metabolic	0.453248	9	**#1**
	e-mail	0.582828	10	**#1**
	polblogs	0.427105	278	**#1**
	power	0.940850	43	**#1**
	PGPgiantcompo	0.886081	111	#2
	astro-ph	0.74462	1083	**#1**
	memplus	0.695284	64	#3
	as-22july06	0.677575	42	#2
	cond-mat-2005	0.745064	1902	#2
	kron_g500-simple-logn16	0.065055	10027	**#1**
Qu2	preferentialAttachment	0.315993	9	**#1**
	smallworld	0.793041	242	**#1**
	G_n_pin_pout	0.499344	171	#2
	luxembourg.osm	0.98962	275	**#1**
	rgg_n_2_17_s0	0.978323	133	**#1**
	caidaRouterLevel	0.870905	477	#2
	coAuthorsCiteseer	0.9039	297	#2
	citationCiteseer	0.821744	201	#2
	coPapersDBLP	0.865039	326	#2
	eu-2005	0.941324	389	#2
	audikw1	0.917983	34	**#1**
	ldoor	0.969098	88	#2
	in-2004	0.980537	1637	#2
	belgium.osm	0.994761	580	#4
	333SP	0.988365	229	#4
	kron_g500-simple-logn20	0.049376	253416	#2
	cage15	0.343823	648819	#10
	uk-2002	0.990087	45165	#3

search metaheuristic to that problem and it proves to be very effective. For problems with known optimum values, the heuristic always found an optimal solution at least once. For the DIMACS Implementation Challenge, the best know solution was provided for 11 out of 30 instances. Overall, the proposed algorithm obtained the second prize in the modularity Quality challenge and the fifth place in the Pareto challenge.

References

[ACCHLP] D. Aloise, S. Cafieri, G. Caporossi, P. Hansen, L. Liberti, and S. Perron, *Column Generation Algorithms for Exact Modularity Maximization in Networks*, Physical Review E, vol. 82 (2010), no. 046112.

[AK] G. Agarwal and D. Kempe, *Modularity-maximizing graph communities via mathematical programming*, Eur. Phys. J. B **66** (2008), no. 3, 409–418, DOI 10.1140/epjb/e2008-00425-1. MR2465245 (2009k:91130)

[AY] C.J. Alpert and S.-Z. Yao, *Spectral Partitioning: The More Eigenvectors, The Better*, Proc. 32nd ACM/IEEE Design Automation Conf., 1995, 195–200.

[BC] M.J. Barber and J.W. Clark, *Detecting network communities by propagating labels under constraints*, Physical Review E, vol. 80 (2009), no. 026129.

[BGLL] V. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, *Fast unfolding of communities in large networks*, Journal of Statistical Mechanics, (2008), p. P10008.

[BILPR] S. Boccaletti, M. Ivanchenko, V. Latora, A. Pluchino, and A. Rapisarda, *Detecting complex network modularity by dynamical clustering*, Physical Review E, vol. 75 (2007), 045102(R).

[CFS] D. Chen, R. Fu, and M. Shang, *A fast and efficient heuristic algorithm for detecting community structures in complex networks*, Physica A, vol. 388 (2009), 2741–2749.

[CHL] S. Cafieri, P. Hansen, and L. Liberti, *A locally optimal heuristic for modularity maximization of networks*, Physical Review E, vol. 83 (2011), 056105(1-8).

[CNM] A. Clauset, M. Newman, M., and C. Moore, *Finding community structure in very large networks*, Physical Review E, vol. 70 (2004), no. 066111.

[D] H. Djidjev, *A scalable multilevel algorithm for graph clustering and community structure detection*, LNCS, vol. 4936, (2008).

[DA] J. Duch, and A. Arenas, *Community identification using extremal optimization*, Physical Review E 72 (2005) 027104(2).

[DDA] L. Danon, A. Diaz-Guilera, and A. Arenas, *The effect of size heterogeneity on community identification in complex networks*, Journal of Statistical Mechanics, vol. P11010, 2006.

[DGSW] D. Delling, R. Görke, C. Schulz, D. Wagner. *ORCA reduction and contraction graph clustering*. In 5th Int. Conf. on Algorithmic Aspects in Information and Management (AAIM), 2009, 152–165.

[F] Santo Fortunato, *Community detection in graphs*, Phys. Rep. **486** (2010), no. 3-5, 75–174, DOI 10.1016/j.physrep.2009.11.002. MR2580414 (2011d:05337)

[FLZWD] Y. Fan, M. Li, P. Zhang, J. Wu, Z. Di, *Accuracy and precision of methods for community identification in weighted networks*, Physica A, vol. 377 (2007), 363–372.

[GA] R. Guimerà and A. Amaral, *Functional cartography of complex metabolic networks*, Nature, vol. 433 (2005), pp. 895–900.

[GH] Olivier Goldschmidt and Dorit S. Hochbaum, *A polynomial algorithm for the k-cut problem for fixed k*, Math. Oper. Res. **19** (1994), no. 1, 24–37, DOI 10.1287/moor.19.1.24. MR1290008 (95h:90154)

[GN] M. Girvan and M. E. J. Newman, *Community structure in social and biological networks*, Proc. Natl. Acad. Sci. USA **99** (2002), no. 12, 7821–7826 (electronic), DOI 10.1073/pnas.122653799. MR1908073

[HJ] Pierre Hansen and Brigitte Jaumard, *Cluster analysis and mathematical programming*, Math. Programming **79** (1997), no. 1-3, Ser. B, 191–215, DOI 10.1016/S0025-5610(97)00059-2. Lectures on mathematical programming (ismp97) (Lausanne, 1997). MR1464767 (98g:90043)

[HMP] Pierre Hansen, Nenad Mladenović, and José A. Moreno Pérez, *Variable neighbourhood search: methods and applications*, 4OR **6** (2008), no. 4, 319–360, DOI 10.1007/s10288-008-0089-1. MR2461646 (2009m:90198)

[JMF] A. Jain, M. Murty, and P. Flynn, *Data clustering: A review*, ACM Computing Surveys, vol. 31 (1999), 264–323.

[KR] Leonard Kaufman and Peter J. Rousseeuw, *Finding groups in data*, Wiley Series in Probability and Mathematical Statistics: Applied Probability and Statistics, John Wiley & Sons Inc., New York, 1990. An introduction to cluster analysis; A Wiley-Interscience Publication. MR1044997 (91e:62159)

[KSKK] J. Kumpula, J. Saramaki, K. Kaski, and J. Kertesz, *Limited resolution and multiresolution methods in complex network community detection*, Fluctuation and Noise Letters, vol. 7 (2007), L209–L214.

[LH] S. Lehmann and L. Hansen, *Deterministic modularity optimization*, European Physical Journal B, vol. 60 (2007), 83–88.

[LM] X. Liu and T. Murata, *Advanced modularity-specialized label propagation algorithm for detecting communities in networks*, Physica A, vol. 389 (2010), 1493–1500.

[LS] W. Li and D. Schuurmans, *Modular Community Detection in Networks*, In Proc. of the 22nd. Intl Joint Conf. on Artificial Intelligence, 2011, 1366–1371.

[M] Boris Mirkin, *Clustering for data mining*, Computer Science and Data Analysis Series, Chapman & Hall/CRC, Boca Raton, FL, 2005. A data recovery approach. MR2158827 (2006d:62002)

[MAD] A. Medus, G. Acuna, and C. Dorso, *Detection of community structures in networks via global optimization*, Physica A, vol. 358 (2005), 593–604.

[MD] Jonathan P. K. Doye and Claire P. Massen, *Self-similar disk packings as model spatial scale-free networks*, Phys. Rev. E (3) **71** (2005), no. 1, 016128, 12, DOI 10.1103/PhysRevE.71.016128. MR2139325 (2005m:82056)

[MHSWL] J. Mei, S. He, G. Shi, Z. Wang, and W. Li, *Revealing network communities through modularity maximization by a contraction-dilation method*, New Journal of Physics, vol. 11 (2009), 043025.

[N04] M. Newman, *Fast algorithm for detecting community structure in networks*, Physical Review E, vol. 69 (2004), 066133.

[N06] M. Newman, *Modularity and community structure in networks*, In Proc. of the National Academy of Sciences, 2006, 8577–8582.

[NG] M. Newman and M. Girvan, *Finding and evaluating community structure in networks*, Physical Review E, vol. 69 (2004), 026133.

[NHZW] Yan Qing Niu, Bao Qing Hu, Wen Zhang, and Min Wang, *Detecting the community structure in complex networks based on quantum mechanics*, Phys. A. **387** (2008), no. 24, 6215–6224, DOI 10.1016/j.physa.2008.07.008. MR2591580 (2010k:81089)

[NR] A. Noack and R. Rotta, *Multi-level algorithms for modularity clustering*, LNCS vol. 5526 (2009), 257–268.

[RMP] T. Richardson, P. Mucha, and M. Porter, *Spectral tripartitioning of networks*, Physical Review E, vol. 80 (2009), 036111.

[RZ] J. Ruan and W. Zhang, *Identifying network communities with a high resolution*, Physical Review E, vol. 77 (2008), 016104.

[SC] P. Schuetz and A. Caflisch, *Efficient modularity optimization by multistep greedy algorithm and vertex mover refinement*, Physical Review E, vol. 77 (2008), 046112.

[SDJB] Y. Sun, B. Danila, K. Josic, and K.E. Bassler, *Improved community structure detection using a modified fine-tuning strategy*, Europhysics Letters, vol. 86 (2009), 28004.

[SM] J. Shi and J. Malik, *Normalized cuts and image segmentation*, IEEE TPAMI, vol. 22 (2000), 888–905.

[THB] M. Tasgin, A. Herdagdelen, and H. Bingol, *Community detection in complex networks using genetic algorithms*, arXiv:0711.0491, (2007).

[WT] K. Wakita and T. Tsurumi, *Finding community structure in mega-scale social networks*, Tech. Rep. 0702048v1, arXiv, 2007.

[XTP] G. Xu, S. Tsoka, and L. Papageorgiou, *Finding community structures in complex networks using mixed integer optimization*, The European Physical Journal B, vol. 60 (2007), 231–239.

Universidade Federal do Rio Grande do Norte, Campus Universitário s/n, Natal-RN, Brazil, 59072-970
E-mail address: aloise@dca.ufrn.br

GERAD & HEC Montréal, 3000, chemin de la Côte-Sainte-Catherine, Montréal (Québec) Canada, H3T 2A7
E-mail address: gilles.caporossi@hec.ca

GERAD & HEC Montréal, 3000, chemin de la Côte-Sainte-Catherine, Montréal (Québec) Canada, H3T 2A7
E-mail address: pierre.hansen@hec.ca

LIX, École Polytechnique, F-91128 Palaiseau, France
E-mail address: liberti@lix.polytechnique.fr

GERAD & HEC Montréal, 3000, chemin de la Côte-Sainte-Catherine, Montréal (Québec) Canada, H3T 2A7
E-mail address: sylvain.perron@hec.ca

INP-Grenoble, 46, avenue Félix Viallet, 38031 Grenoble Cedex 1, France
E-mail address: manuel.ruiz@grenoble-inp.fr